

# LeetCode 2363: Merge Similar Items

## Algorithm & Approach

The problem requires us to merge two 2D arrays, `items1` and `items2`, where each element is an array containing an item ID and a weight. If an item ID exists in both arrays, their weights must be summed up. The final output must be sorted in ascending order by item ID.

### Step-by-Step Logic:

- **Data Structure Selection:** An `unordered_map<int, int>` is chosen because it allows  $O(1)$  average time complexity for both insertions and lookups. The key stores the *item ID* and the value stores the *cumulative weight*.
- **First Pass (Iterating over `items1`):** Traverse the first array and populate the map. We store the weight of each item against its corresponding ID.
- **Second Pass (Iterating over `items2`):** Traverse the second array. For each item:
  - If its ID is already present in the map, add the new weight to the existing weight.
  - If it is not present, insert the new ID-weight pair into the map.
- **Result Extraction:** Iterate over all key-value pairs in the map, create pairs of {ID, Weight}, and push them into the result vector `ans`.
- **Sorting:** Finally, sort the `ans` vector. By default, sorting a 2D vector in C++ sorts based on the first element of each inner vector (the ID), which satisfies the problem constraints.

## Source Code implementation

```
class Solution {
public:
    vector<vector<int>> mergeSimilarItems(vector<vector<int>>& items1,
vector<vector<int>>& items2) {

        unordered_map<int,int> mp;
        vector<vector<int>> ans;

        int n=items1.size();
        int m=items2.size();

        for(int i=0;i<n;i++){
            mp[items1[i][0]]=items1[i][1]; // nums1 ke saare item daal do
map me
        }

        for(int j=0;j<m;j++){
            if(mp.find(items2[j][0])!=mp.end()){ // agar pehle bhi same id
ka item dekh rkha he to add krna padega weight
                mp[items2[j][0]]+=items2[j][1];
                continue;
            }

            mp[items2[j][0]]=items2[j][1];

        }

        for(auto it:mp){
            int val=it.first;
            int w=it.second;

            ans.push_back({val,w});
        }

        sort(ans.begin(),ans.end());

        return ans;
    }
};
```

## Complexity Analysis

- **Time Complexity:**  $O(N + M + K \log K)$

Where  $N$  is the size of `items1`,  $M$  is the size of `items2`, and  $K$  is the number of unique item IDs. We iterate over both arrays once, which takes  $O(N + M)$  time. Sorting the final answer array takes  $O(K \log K)$  time.

- **Space Complexity:**  $O(K)$

We use a hash map to store unique items and their cumulative weights. In the worst case, all items have distinct IDs, meaning the map will store exactly  $K = N + M$  entries. The space required for the `ans` vector is also  $O(K)$ .